Cryptographic Engineering 2026 - Final Project

September 2026

Project Part A Deadline (Code): November 15, 2026, 23:59 (Taipei time)
Final Submission Deadline (Code & Report): December 27, 2026, 23:59 (Taipei time)
Submission via NTU COOL!
To be completed individually.

Assignment

Optimize cryptographic algorithms for Arm Cortex-M4 microcontrollers.
Download the starter code repository from: https://github.com/mkannwischer/ce2026

The repository contains four projects in two parts. **Part A** is classical cryptography, SHAKE256 and ECDH25519; **Part B** is post-quantum cryptography, ML-KEM and ML-DSA. You pick **one project from each part** and optimize it for the STM32F407, writing assembly where it pays off. Both parts are described in detail under Project Overview below.

There are two code deadlines. By **November 15** your Part A project has to meet the minimum requirements. No report is required at that deadline, and it is worth **20% of your course grade**. By **December 27** you submit both projects and your report, and the quality of both is graded. That submission is worth **30% of your course grade**.

Finally, you present your work in class on **December 15** (or potentially December 8, depending on the number of students). The presentation is worth **20% of your course grade**, your slides are due the day before, and there will be questions about your code and your report.

Your project does not have to be finished by the time you present. The code deadline is December 27, after the presentations. If parts of your work are still incomplete, present what you have together with a plan for the rest.

Grading & Submission

Grading Breakdown

Final project = 70% of course grade:

- 20%: Part A meeting the minimum requirements by November 15
- 30%: implementation quality of *both* parts & final report
- 20%: Final presentation

Submission Requirements

There are three submissions, all via NTU COOL:

- **Project Part A**, November 15: an archive (`zip`, `tar.bz2`) containing your Part A code. No report is required.

- **Presentation slides**, the day before you present: your slides as a `pdf`.
- **Final submission**, December 27: an archive containing your complete code for both parts, and a `pdf` report of at most 20 pages describing your optimization techniques and results for both parts.

Hardware

Each student receives an STM32F407 development board on loan, for testing their optimized implementations on real hardware. To be eligible for a board, you must successfully complete Homework 0, which demonstrates that your development environment is working.

Boards are handed out from **Week 4 (2026-09-29)** against a **deposit of TWD 500**, which is refunded in full when the board is returned.

The board must be returned on **Tuesday 2027-01-05**, in the lecture slot and in the same room. If you have already finished, you may instead return it at the final presentations on 2026-12-15. If you cannot make either date, email ce2026@kannwischer.eu to arrange a time.

Important Notes

- All tests must pass on both QEMU and STM32F407 hardware
- Do not modify the provided tests in `test.c` (you may add additional benchmarks or tests)
- Final presentations will include questions about your implementation

Project Overview

The repository contains four cryptographic reference implementations organized into two parts:

Part A: Classical Cryptography

- **SHAKE256**: Extensible-output hash function based on the Keccak permutation (FIPS202)
- **ECDH25519**: Elliptic curve Diffie-Hellman key exchange using Curve25519

Part B: Post-Quantum Cryptography

- **ML-KEM**: Module-Lattice-based Key Encapsulation Mechanism (FIPS203)
- **ML-DSA**: Module-Lattice-based Digital Signature Algorithm (FIPS204)

Completion Requirements: complete 1 project from Part A and 1 project from Part B.

Code Requirements

In order to get a passing grade for the Cryptographic Engineering course, the code you submit must fulfill **all the following minimal requirements**:

1. It must not have any secret-dependent branches or access to memory at secret-dependent locations (timing attack resistance).
2. The submitted software must offer the same functionality as the reference implementations, i.e., all tests must pass.
3. The assembly-optimized functions must be faster than the C reference implementations.
4. Meet the specific minimum requirements for each project listed below.

SHAKE256

- Write the Keccak-$f[1600]$ permutation in assembly
- Permutation (binary) code size limit: 4 096 bytes (no round unrolling)
- Must be faster than C reference implementation

ECDH25519

- $<$ 35000000 cycles for ECDH `scalarmult_base` and $<$ 32000000 cycles for ECDH `scalarmult`

ML-KEM

- Write the Number Theoretic Transform (NTT) in assembly
- NTT code size limit: 1 024 bytes (excluding twiddle factors)
- Write at least two other polynomial functions in assembly
- Each assembly function must achieve speedup over C reference

ML-DSA

- Optimize total stack usage of ML-DSA-65 to at most 32 KiB for all operations
- Write at least 2 polynomial functions in assembly
- Each assembly function must achieve speedup over C reference

Evaluation Criteria

Each project has different optimization priorities for achieving a good grade:
SHAKE256

- Primary: Speed (performance improvement over C reference)
- Constraint: Must stay under 4,096 byte code size limit
- Best grades: Maximum speed while remaining under code size limit

ECDH25519

- Primary: Speed
- Best grades: Fastest implementations

ML-KEM

- NTT: Minimize code size + achieve speedup
- Other functions: Speed (performance improvement over C reference)
- Best grades: Compact NTT with good performance + fast other operations

ML-DSA

- NTT: Speed
- Primary: Minimize stack usage
- Speed can be sacrificed for lower stack usage
- Best grades: Lowest stack usage

To obtain a good grade, a comparison with the state-of-the-art will be required for the completed parts.

AI Tools Policy

AI tools are **permitted** with transparency requirements:

- Document exactly how AI was used
- AI-generated code must be understood, tested, and validated by you
- You must be able to explain every part of your code
- Direct copy-paste without understanding is prohibited

Getting Started

Development Environment

Linux (Ubuntu recommended) or macOS. Windows users should work inside an Ubuntu virtual machine, for example under VirtualBox.

- **Arm GCC toolchain**: `sudo apt install gcc-arm-none-eabi` (Ubuntu/Debian) or `brew install arm-non` (macOS)
- **QEMU with Arm support**: `sudo apt install qemu-system-arm` (Ubuntu/Debian) or `brew install qemu` (macOS)
- **STM32F407 flashing**: `sudo apt install stlink-tools` (Ubuntu/Debian) or `brew install stlink` (macOS)

First Build

Clone the starter repository and run the test framework in QEMU:

```
git clone --recursive https://github.com/mkannwischer/ce2026.git
cd ce2026/shake256
make run-qemu PLATFORM=qemu
```

Make Targets

- `make PLATFORM=qemu` : build for QEMU emulation
- `make PLATFORM=stm32` : build for the STM32F407
- `make run-qemu PLATFORM=qemu` : run the tests in QEMU
- `make flash-stm32 PLATFORM=stm32` : flash to the development board
- `make size PLATFORM=qemu` : measure binary code size

Getting Help

There will be time to work on the project during class, particularly in the second half of the course. That is the best opportunity to ask questions.

You can also email ce2026@kannwischer.eu at any point.

Optimization Hints

SHAKE256

- Focus on the Keccak permutation: this is the performance bottleneck
- Consider bit-interleaved representation for efficient rotation operations
- The permutation does not need to be performed in-place
- Avoid unrolling multiple rounds to stay within code size limits

ECDH25519

Before optimizing, investigate and eliminate timing leaks (secret-dependent branches/memory access). Optimization opportunities include:

- Optimized field arithmetic using larger radix (e.g., radix-$2^{25.5}$)
- Specialized base-point scalar multiplication
- Efficient group arithmetic and point representations
- Montgomery ladder for scalar multiplication

ML-KEM

- NTT and Keccak are the most critical function for performance
- Consider polynomial multiplication, addition/subtraction operations
- Look at compression/decompression functions for additional speedup
- Profile the code to identify other bottlenecks
- You may want to use a fast Keccak implementation available in the literature (no code size constraints)

ML-DSA

- Use stack measurement tools to identify high stack usage functions
- Consider in-place operations and buffer reuse
- Optimize polynomial operations for both speed and stack usage
- The NTT and polynomial arithmetic are good assembly targets
- You may want to use a fast Keccak implementation available in the literature (no code size constraints)